

Resumo prova 1

Conversão de hexadecimal para binário:

```
Converte o binário 110101101 para hexadecimal:  
- Bloco da direita: 1101  
- Bloco do meio: 0110  
- Bloco da esquerda (incompleto): 1 -> completando com zeros fica 0001  
- Resultado dos grupos: 0001 | 0110 | 1101  
- Resultado final: 16D em hexadecimal.
```

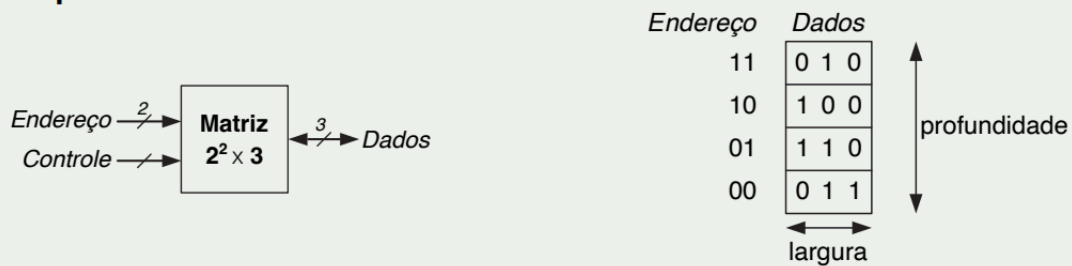
Relembrando complemento de dois:

```
1. Converta a sua representação positiva (magnitudo) para binário  
2. Inverta todos os bits (complemento de um)  
3. Some um (1)  
Exemplo: Converter -7 considerando uma representação de 5 bits.  
1 +7 = 00111  
2 11000  
3 11000 + 1 = 11001  
Assim, -7 = 11001
```

Memória RAM e ROM:

- As memória possuem três barramentos
 - Endereço: Posição que será operada
 - Dados: Transição dos dados (leitura e escrita)
 - Controle: Sinal que faz a leitura e/ou escrita de dados
- Uma memória de N bits de endereço e M bits de dados
 - Possui 2^N linhas e M colunas
 - Profundidade: Número de palavras = 2^N
 - Largura: Tamanho da palavra em bits = M
 - Tamanho da memória: $(2^N) * M$

■ Exemplo 1: Memória 4x3



■ **Profundidade** = $N = 2$, então $2^2 = 4$ palavras.

■ **Largura** = $M = 3$, então a largura de cada palavra é 3 bits.

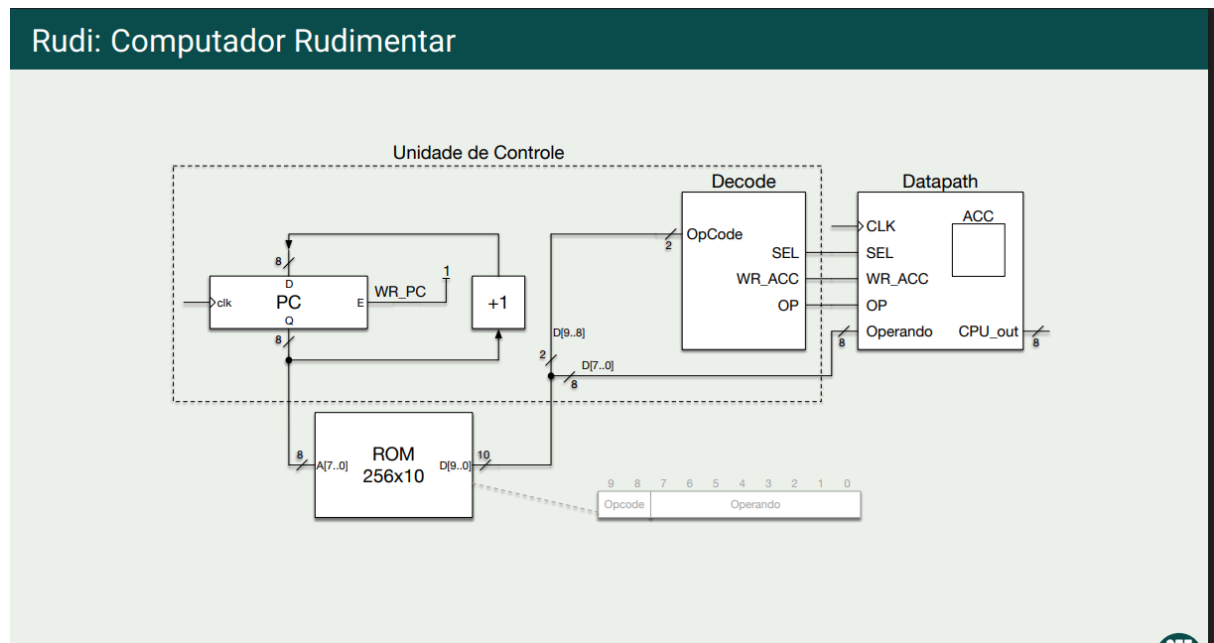
■ **Tamanho da memória** = $4 \times 3 = 12 \text{ bits}$.

Tipos de memórias:

- Ram (Random Access Memory):
 - Perde dados quando a energia é desligada
 - Leitura e escrita rápida
 - Principal memória do computador
 - DRAM: usa um capacitor, o descarregamento do capacitor causa degradação no valor. A leitura destrói o valor armazenado
 - SRAM: usa duas portas inversoras realimentando, mais simples de controlar que DRAM
- Rom (Ready Only Memory):
 - Não volátil: retém os dados quando não há energia
 - Lê rapidamente, mas a gravação é impossível ou muito lenta
 - Câmeras, celulares, pen drives são todas ROMs
 - As ROM reprogramáveis fornecem um mecanismo reversível para ligar ou desligar o transístor.
 - A Erasable PROM (EPROM) pode ser apagada utilizando luz UV. A EEPROM e a Flash são apagáveis eletricamente por circuitos incluídos junto ao chip, assim não é necessário utilizar luz UV para apagar.
 - As células de bit EEPROM são individualmente apagáveis. Já a Flash apaga blocos maiores de bits e é mais barata porque são necessários

menos circuitos de apagamento.

Exemplo com RUDI:



- Para executar o cálculo $5 + 3 + 7 + 8 + 16 - 2 - 1$, a memória deveria ser carregada com os seguintes valores:

Operação	Endereço	Conteúdo
$ACC \leftarrow 5$	0x00	01 00000101
$ACC \leftarrow ACC + 3$	0x01	10 00000011
$ACC \leftarrow ACC + 7$	0x02	10 00000111
$ACC \leftarrow ACC + 8$	0x03	10 00001000
$ACC \leftarrow ACC + 16$	0x04	10 00010000
$ACC \leftarrow ACC - 2$	0x05	11 00000010
$ACC \leftarrow ACC - 1$	0x06	11 00000001
$CPU_out \leftarrow ACC$	0x07	00 XXXXXXXX

9	8	7	6	5	4	3	2	1	0
OpCode		Operando							

Arquitetura: Interface mais abstrata e com menos detalhes, ou seja, a interface do programador

Organização: Mais detalhes, constitui-se na implementação da arquitetura em si

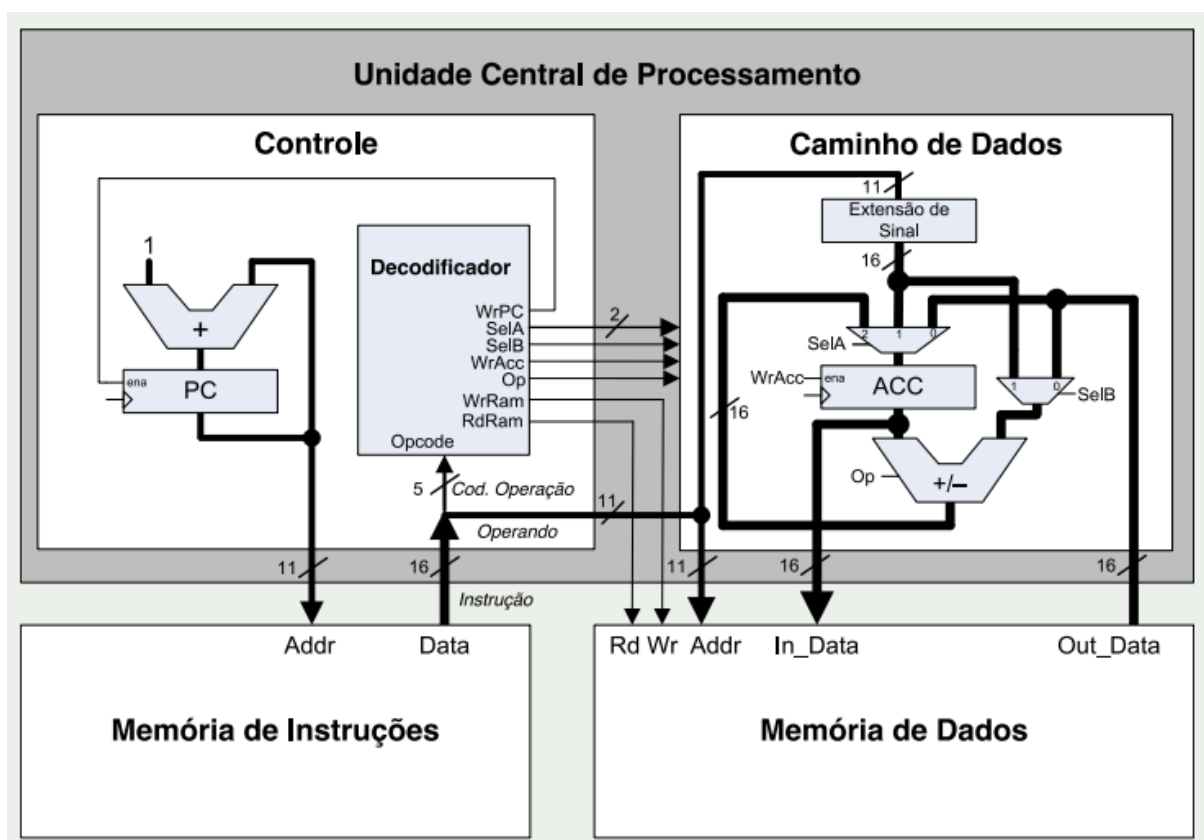
Atributos arquiteturais de um processador:

- Tamanho da palavra de dados: Número de bits do dado manipulado pelo processador
- Tipo de dados: Se é inteiro (com ou sem sinal), real, float...
- Tamanho da palavra de instrução: Número de bits usados para representar uma instrução de programa
- Formato das instruções: Como os bits devem ser organizados para informar o processador o que deve fazer
- Modos de endereçamento: Método de acesso aos dados processados pelas instruções
- Registradores: Unidades de armazenamento interno
- Conjunto de instruções: Tudo que o processador pode fazer e seus códigos

Exemplo de processador: BIP (Basic instruction-set Processor):

- Tamanho de palavra de 16 bits e com inteiros com sinal, operação com 5 bits e 11 de operando. Utiliza um acumulador para registrar e processar novas entradas.

Organização do BIP 1:



Arquitetura do BIP II:

- Inclui um registrador com dois bits de estado:
 - Status.Z (Zero): Resultado da última operação aritmética = 0
 - Status.N (Negative): Resultado da última operação aritmética < 0
- Inclui duas classes adicionais de instrução:
 - Desvio incondicional: Muda o fluxo de execução, atualizando o PC com o endereço de instrução especificado
 - Desvio condicional: Se a condição especificada for verdadeira, desvia para o endereço de instrução especificado
- Além das instruções do BIP I também há as novas

Instrução de desvio incondicional		
JMP	Jump	Desvia incondicionalmente
Instruções de desvio condicional		
BEQ	Branch on Equal	Desvia se igual
BNE	Branch on Not Equal	Desvia se não igual
BGT	Branch on Greater Than	Desvia se maior que
BGE	Branch on Greater or Equal	Desvia se maior ou igual que
BLT	Branch on Less Than	Desvia se menor que
BLE	Branch on Less or Equal	Desvia se menor ou igual que

Controle de desvio do BIP II:

Sinais de Entrada (recebidos pelo Decodificador)

- **OpCode:** Código de operação de 5 bits retirado da instrução buscada na memória, que informa à unidade de controle qual instrução deve ser executada.
- **N (Negative):** Bit de estado (*flag*) que indica se o resultado da última operação aritmética executada na ULA foi menor que zero. Este valor é extraído diretamente do bit mais significativo (bit 15) da saída da ULA.
- **Z (Zero):** Bit de estado que indica se o resultado da última operação aritmética na ULA foi igual a zero. É gerado a partir de uma porta NOR

conectada a todos os bits de saída da ULA.

Sinais de Controle (gerados pelo Decodificador)

- **WrPC (Branch):** Sinal que atua no controle de desvio do *Program Counter* (PC). Ele comanda o multiplexador antes do PC: se ativado, o PC recebe o endereço de desvio (Operando da instrução); se desativado, recebe `PC + 1` para continuar o fluxo linear do programa.
- **SelA[1..0]:** Barramento de seleção de 2 bits que controla o multiplexador conectado à entrada do Acumulador (ACC). Ele define se o ACC receberá o resultado da ULA, um dado lido da Memória de Dados, ou o operando com extensão de sinal.
 - Esse sinal decide o que vai ser salvo dentro do acumulador após o pulso de clock
- **SelB:** Sinal de seleção que controla o multiplexador da entrada B da ULA, definindo se a operação matemática usará um dado proveniente da memória (`In_Data`) ou o operando da instrução atual.
 - Decide quem será o segundo operador da entrada B da ULA
- **WrAcc (Write ACC):** Sinal de *enable* que autoriza a gravação de um novo dado no registrador Acumulador.
- **Op (Operation):** Define a operação matemática que a Unidade Lógica Aritmética (ULA) deverá executar. Geralmente, `0` configura a ULA para soma e `1` configura para subtração.
- **WrNZ (Write N/Z):** Sinal de *enable* que autoriza a atualização dos flip-flops que guardam o estado das *flags* `N` e `Z`. É ativado apenas durante execuções de instruções aritméticas.
- **WrRam (Write RAM):** Habilita a operação de escrita na Memória de Dados, comandando a gravação do valor contido no ACC em um endereço de memória (usado pela instrução `STO`).
- **RdRam (Read RAM):** Habilita a operação de leitura na Memória de Dados, permitindo que os dados armazenados nela fluam para o Caminho de Dados (`In_Data`).

Tabela de DECODE opções:

Mnemônico	Opcode [4..0]				N	Z	WrPC	SelA1	SelA0	SelB	WrAcc	OpSel	RdRam	WrRam	Branch	WrNZ
HLT	0	0	0	0	X	X	0	X	X	X	0	X	1	0	0	0
STO	0	0	0	1	X	X	1	X	X	X	0	X	0	1	0	0
LD	0	0	1	0	X	X	1	0	0	X	1	X	1	0	0	0
LDI	0	0	1	1	X	X	1	0	1	X	1	X	1	0	0	0
ADD	0	1	0	0	X	X	1	1	0	0	1	0	1	0	0	1
ADDI	0	1	0	1	X	X	1	1	0	1	1	0	1	0	0	1
SUB	0	1	1	0	X	X	1	1	0	0	1	1	1	0	0	1
SUBI	0	1	1	1	X	X	1	1	0	1	1	1	1	0	0	1
BEQ	1	0	0	0	X	0	1	X	X	X	0	X	1	0	0	0
	1	0	0	0	X	1	1	X	X	X	0	X	1	0	1	0
BNE	1	0	0	1	X	0	1	X	X	X	0	X	1	0	1	0
	1	0	0	1	X	1	1	X	X	X	0	X	1	0	0	0
BGT	1	0	1	0	0	0	1	X	X	X	0	X	1	0	1	0
	1	0	1	0	0	1	1	X	X	X	0	X	1	0	0	0
	1	0	1	0	1	0	1	X	X	X	0	X	1	0	0	0
	1	0	1	0	1	1	1	X	X	X	0	X	1	0	0	0
BGE	1	0	1	1	0	0	1	X	X	X	0	X	1	0	1	0
	1	0	1	1	0	1	1	X	X	X	0	X	1	0	1	0
	1	0	1	1	1	0	1	X	X	X	0	X	1	0	0	0
	1	0	1	1	1	1	1	X	X	X	0	X	1	0	0	0
BLT	1	1	0	0	0	0	1	X	X	X	0	X	1	0	0	0
	1	1	0	0	0	1	1	X	X	X	0	X	1	0	0	0
	1	1	0	0	1	0	1	X	X	X	0	X	1	0	1	0
	1	1	0	0	1	1	1	X	X	X	0	X	1	0	1	0
BLE	1	1	0	1	0	0	1	X	X	X	0	X	1	0	0	0
	1	1	0	1	0	1	1	X	X	X	0	X	1	0	1	0
	1	1	0	1	1	0	1	X	X	X	0	X	1	0	1	0
JMP	1	1	1	0	X	X	1	X	X	X	0	X	1	0	1	0